INT234: PREDICTIVE ANALYTICS

PROJECT REPOR

IPO LISTING STATUS PREDICTION USING MACHINE LEARNING CLASSIFICATION

Submitted by

AMBATI SAKETH REDDY

Registration No.: 12405720

Programme: B.Tech Computer Science and Engineering

Course Code: INT234

Under the Guidance of

DR. MRINALINI RANA

U.ID: 22138

Discipline of CSE/IT

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab





ABSTRACT

The project titled **"IPO Listing Status Prediction Using Machine Learning Classification"** focuses on analysing Initial Public Offering (IPO) records and predicting their recorded listing status using machine learning classification techniques.

The project uses an IPO dataset containing information such as IPO type, year, number of news reports, amount status, country, sector and listing status. Data preprocessing techniques are applied to handle missing values, check duplicate records, convert date information, extract relevant features, encode categorical variables and scale numerical features.

Exploratory Data Analysis (EDA) is performed to understand the distribution of IPO listing statuses, year-wise IPO records, country-wise IPO distribution and sector-wise IPO distribution. Four machine learning algorithms are implemented: Logistic Regression, K-Nearest Neighbors, Decision Tree and Random Forest.

The models are trained using 80% of the dataset and tested using the remaining 20%. Their performance is evaluated and compared using accuracy, macro precision, macro recall and macro F1-score. A confusion matrix is also used to analyse the classification results across different listing-status categories.

The project demonstrates the application of machine learning classification techniques to IPO-related data and identifies patterns in recorded listing statuses. It also highlights the importance of data preprocessing, exploratory data analysis and model evaluation in developing a classification system.

Keywords: Initial Public Offering (IPO), Machine Learning, Classification, Exploratory Data Analysis, Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest.

DECLARATION

I, **AMBATI SAKETH REDDY**, student of Bachelor of Technology in Computer Science and Engineering under the Discipline of CSE/IT at Lovely Professional University, Phagwara, Punjab, hereby declare that the project report entitled **"IPO LISTING STATUS PREDICTION USING MACHINE LEARNING CLASSIFICATION"** is based on my own work carried out as part of the Data Science Minor Project.

I declare that the work presented in this report is based on the study, implementation, analysis and experimentation performed by me under the guidance of my faculty coordinator. The project involves the use of Python programming, data preprocessing, exploratory data analysis and machine learning classification techniques to analyze IPO records and predict their listing status.

I further declare that the information provided in this report is genuine to the best of my knowledge and that all external sources, references and materials used during the project have been appropriately acknowledged.

Date:

Place: Phagwara, Punjab

Signature: Saketh

Name: AMBATI SAKETH REDDY

Registration No.:12405720

CERTIFICATE

This is to certify that **AMBATI SAKETH REDDY**, bearing **Registration No. 12405720**, has successfully completed the Data Science Minor Project titled **"IPO LISTING STATUS PREDICTION USING MACHINE LEARNING CLASSIFICATION"** under my guidance and supervision.

The project has been carried out as part of the academic requirements of the Bachelor of Technology in Computer Science and Engineering programme at Lovely Professional University, Phagwara, Punjab.

The work presented in this report includes dataset analysis, exploratory data analysis, data preprocessing, implementation of machine learning classification algorithms and evaluation of their performance.

To the best of my knowledge, the present work is the result of the student's original development, effort and study.

Date: 30/10/2026

Signature and Name of the Supervisor

Name: DR. MRINALINI RANA

Designation: Assistant Professor

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab.

Acknowledgement:

I would like to express my sincere gratitude to Lovely Professional University, Phagwara, Punjab, for providing me with the opportunity to undertake this Data Science Minor Project as part of my academic curriculum.

I am deeply thankful to my faculty coordinator, [Faculty Name], for providing valuable guidance, continuous support and encouragement throughout the development of this project. Their suggestions and feedback helped me understand the concepts of data analysis and machine learning and apply them effectively during the implementation of the project.

I would also like to thank the School of Computer Science and Engineering for providing the necessary academic resources and learning environment to complete this project successfully.

I am grateful to the developers and contributors of Python and its supporting libraries, including Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn, which played an important role in data processing, visualization, model development and performance evaluation.

Finally, I would like to thank my family, friends and everyone who directly or indirectly supported me during the completion of this project.

AMBATI SAKETH REDDY

Table of Contents:

Sl.no	Particulars	Page No.
1	Cover Page	1
2	Abstract	2
3	Declaration	3
4	Certificate	4
5	Acknowledgement	5
6	Table of Contents	6
7	List of Figures	7
8	Introduction	7 – 11
9	Source of Dataset	11 – 15
10	Exploratory Data Analysis (EDA)	15 – 22
11	Data Preprocessing and Methodology	22 – 27
12	Implementation and Model Training	27 – 30
13	Results and Performance Analysis	30 – 37
14	Conclusion and Future Scope	37 – 40
15	References	40
16	Appendix A – Python Source Code	41– 48
17	Appendix B – Project Links and Screenshots	48 – 49

List of Figures:

LIST OF FIGURES	
Figure No.	**Figure Description**
Figure 3.4	IPO Listing Status Distribution
Figure 3.5	IPO Records by Year
Figure 3.6	Top 10 Countries by IPO Records
Figure 3.7	Top 10 IPO Sectors
Figure 6.1	Accuracy Comparison
Figure 6.2	Precision Comparison
Figure 6.3	Recall Comparison
Figure 6.4	F1 Score Comparison
Figure 6.5	Confusion Matrix

INTRODUCTION

1.1 Introduction

An Initial Public Offering (IPO) is a process through which a privately held company offers its shares to the public for the first time. It allows companies to raise capital from investors and provides an opportunity for public participation in the company's ownership. IPOs play an important role in financial markets because they help businesses obtain funds for expansion, development, debt repayment and other corporate requirements.

The IPO market generates a large amount of information, including company details, issue information, financial announcements, industry categories, geographical locations and listing updates. This information can be analyzed to understand market activity and identify patterns in IPO records.

With the growth of digital financial information, Data Science has become an important technology for analyzing financial datasets. It provides methods for collecting, processing, visualizing and interpreting large amounts of structured data. Machine Learning further extends these capabilities by allowing computer systems to identify patterns from historical data and generate predictions.

This project, titled "IPO Listing Status Prediction Using Machine Learning Classification", focuses on analyzing historical IPO records and developing classification models to predict the listing status of IPOs using selected features.

The project uses Python programming and various Data Science libraries, including Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn. Four Machine Learning algorithms are implemented: Logistic Regression, K-Nearest Neighbors, Decision Tree and Random Forest.

The performance of these models is compared using evaluation measures such as Accuracy, Precision, Recall and F1-Score. The project also uses Exploratory Data Analysis (EDA) to understand IPO distributions across years, countries, sectors and listing-status categories.

1.2 Background of the Project

Financial markets continuously generate information about companies, investments and public offerings. IPOs are an important part of these markets because they allow companies to access public investment and provide investors with opportunities to participate in business growth.

The listing status of an IPO indicates its progress or outcome in the available dataset. Different records may represent offerings that have been announced or filed, completed, withdrawn or postponed.

Analyzing these categories helps in understanding the distribution of IPO records and identifying patterns across different periods and business sectors.

Traditional analysis methods often require manual examination of individual records. When the dataset contains information from multiple countries, sectors and years, manual analysis can become time-consuming.

Data Science techniques provide an organized approach to examining this information. Data preprocessing helps prepare the dataset, visualization techniques make patterns easier to understand, and Machine Learning algorithms can be used to classify records based on their available characteristics.

This project applies these techniques to historical IPO data to study listing-status patterns and evaluate the performance of different classification algorithms.

1.3 Importance of IPO Data Analysis

IPO data analysis is useful for understanding the characteristics and distribution of public offerings. It provides information about how IPO records are distributed across geographical locations, industries and time periods.

The major importance of IPO data analysis includes:

1. Understanding Market Trends

Analyzing IPO records across different years helps identify periods with higher or lower numbers of recorded public offerings.

2. Country-Wise Analysis

Examining IPO records by country helps understand the geographical distribution of the available dataset.

3. Sector-Wise Analysis

Sector analysis identifies the industries that contribute a larger number of IPO records.

4. Listing Status Analysis

Analyzing listing-status categories helps understand the distribution of announced, completed and withdrawn or postponed IPO records.

5. Data-Driven Decision Support

Machine Learning techniques can identify patterns in historical records and provide classification outputs that support further analysis.

1.4 Role of Data Science in IPO Analysis

Data Science combines statistical techniques, programming, data visualization and Machine Learning to extract useful information from datasets.

In this project, Data Science techniques are applied through the following stages:

- Data collection and loading.
- Dataset inspection and understanding.
- Missing-value and duplicate-value analysis.
- Data preprocessing and feature engineering.
- Exploratory Data Analysis.
- Machine Learning model development.
- Model evaluation and comparison.
- Interpretation of results.

Python is used as the primary programming language because it provides libraries for data manipulation, visualization and predictive modeling.

Pandas is used to load and manipulate the IPO dataset. NumPy supports numerical operations. Matplotlib and Seaborn are used to generate visualizations. Scikit-learn provides the preprocessing tools, classification algorithms and evaluation metrics required for model development.

1.5 Introduction to Machine Learning Classification

Machine Learning is a branch of Artificial Intelligence that enables computer systems to learn patterns from data and use those patterns to perform tasks without explicitly programming every decision.

Classification is a type of supervised Machine Learning in which a model learns from labeled examples and predicts the category of new observations.

In this project, the target variable is listing_status. The model uses selected IPO-related features to predict the corresponding listing-status category.

The project implements four classification algorithms:

Logistic Regression: A supervised classification algorithm used as a baseline model for predicting categorical outcomes.

K-Nearest Neighbors (KNN): A distance-based classification algorithm that predicts a class using the nearest observations in the training dataset.

Decision Tree: A tree-based algorithm that makes predictions through a sequence of decision rules.

Random Forest: An ensemble learning algorithm that combines multiple decision trees to produce a classification result.

The performance of these algorithms is compared to identify the model that performs best under the selected evaluation metrics.

1.6 Problem Statement

The IPO dataset contains information related to public offerings, including IPO type, year, news reports, amount status, country, sector and first report date. Understanding the relationship between these attributes and the listing status requires systematic data analysis.

Manual examination of a large number of IPO records can make it difficult to identify patterns and compare listing-status categories consistently.

Therefore, the main problem addressed by this project is to develop a Machine Learning-based classification approach that analyzes historical IPO records and predicts their listing status using selected structured features.

The project also aims to compare different classification algorithms and evaluate their performance using standard Machine Learning evaluation metrics.

1.7 Objectives of the Project

The major objectives of this project are:

1. To study and analyze historical IPO records.
2. To understand the structure and characteristics of the IPO dataset.
3. To identify missing values and duplicate records.
4. To perform data preprocessing and feature engineering.
5. To conduct Exploratory Data Analysis to identify IPO trends.
6. To analyze IPO distributions across countries, sectors and years.
7. To implement Logistic Regression, KNN, Decision Tree and Random Forest classifiers.
8. To evaluate model performance using Accuracy, Precision, Recall and F1-Score.
9. To compare the implemented Machine Learning models.
10. To identify the best-performing model based on the evaluation results.

1.8 Scope of the Project

The scope of this project is limited to the analysis and classification of historical IPO records available in the selected dataset.

The project focuses on structured IPO information, including IPO type, year, number of news reports, amount status, country, sector and date-related attributes.

Exploratory Data Analysis is performed to understand the distribution of IPO records and identify patterns in the dataset.

Four Machine Learning classification algorithms are implemented and evaluated using a training and testing approach. Their performance is compared using standard classification metrics.

The project can be further extended by adding additional financial information, increasing the dataset size, applying hyperparameter tuning and developing an interactive prediction application.

The model is intended for academic analysis of historical listing-status records. It does not predict IPO share-price returns, investment profitability or future stock-market performance.

SOURCE OF DATASET

2.1 Introduction

A dataset is the foundation of any data science and machine learning project. The quality, structure, and relevance of the dataset directly influence the analysis and performance of machine learning models.

For this project, an IPO-related dataset is used to study the listing status of Initial Public Offerings (IPOs). The dataset contains information about IPO records, including details such as IPO type, year, number of news reports, amount status, country, sector, and listing status.

The dataset is analysed using Python libraries. Before applying machine learning algorithms, the available data is examined to understand its structure, identify missing values, check duplicate records, and determine the relevant attributes.

2.2 Source of Dataset

The dataset used in this project is an IPO tracking dataset containing historical IPO-related records.

Dataset details:

- Dataset name: IPO Tracker Dataset
- File name: ipo_data_rows.csv
- Data format: CSV (Comma-Separated Values)
- Data domain: Finance and IPO market information
- Project application: IPO listing status classification

Original dataset source: Kaggle

Dataset URL: https://www.kaggle.com/datasets

The CSV file was imported into the Python environment using the Pandas library. The dataset was then examined to understand its structure and identify the attributes required for analysis and model development.

2.3 Overview of the Dataset

The dataset contains records related to Initial Public Offerings from different countries and business sectors. It provides information that can be used to examine IPO-related patterns and classify the listing status recorded in the dataset.

The dataset includes both categorical and numerical attributes. Categorical attributes describe groups or classifications, such as country, sector, IPO type, and amount status. Numerical attributes represent measurable information, such as year and number of news reports.

The dataset was loaded into Python using the Pandas library. The shape, head(), and info() functions were used to inspect the number of records, preview the data, and understand the available columns and their data types.

Dataset dimensions: (3201 Rows, 13 Columns)

2.4 Description of Dataset Attributes

The following attributes are considered in the project.

Attribute	Description	Data category
ipo_type	Represents the type or category of IPO	Categorical
year	Represents the year associated with the IPO record	Numerical
news_reports	Represents the number of news reports associated with the record	Numerical
amount_status	Represents the available amount-related status or classification	Categorical
country	Represents the country associated with the IPO record	Categorical
sector	Represents the business sector associated with the IPO	Categorical
first_report_date	Represents the date of the first report associated with the record	Date
listing_status	Represents the recorded status of the IPO	Target variable

The descriptions above explain the role of each field based on its name and use in the project. The exact meaning of individual values should be interpreted according to the original dataset documentation.

2.5 Target Variable

The target variable selected for this project is listing_status.

The target variable represents the recorded listing status of an IPO. It is used by the machine learning classification algorithms to learn patterns from the input attributes and classify records into the available status categories.

The project uses a supervised machine learning approach because the dataset contains an existing target label.

The exact target categories are taken from the values available in the dataset. These categories are examined during data exploration before model training.

2.6 Input Features Selected for Model Development

The following input features are selected to train and evaluate the classification models:

1. ipo_type
2. year
3. news_reports
4. amount_status
5. country
6. sector
7. report_month
8. report_quarter

The report_month and report_quarter attributes are derived from first_report_date during preprocessing. They provide additional time-related information that can be used by the classification models.

The listing_status column is treated as the target variable and is not included among the input features.

2.7 Classification of Dataset Attributes

For machine learning model development, the selected input attributes are divided into numerical and categorical groups.

Numerical features:

- year
- news_reports
- report_month
- report_quarter

Categorical features:

- ipo_type

- amount_status
- country
- sector

Numerical features are processed using median-based missing-value imputation and standardisation. Categorical features are processed using most-frequent-value imputation and one-hot encoding.

This separation allows the preprocessing steps to be applied appropriately to different types of data.

2.8 Tools and Software Used

The project is developed using Python in the Spyder Integrated Development Environment (IDE). The following libraries are used:

Python: The main programming language used for data analysis, preprocessing, visualisation, and machine learning.

Spyder IDE: Used to write, execute, test, and manage the Python code.

Pandas: Used for loading CSV files, examining records, handling data, and performing data operations.

NumPy: Used for numerical operations and data handling.

Matplotlib: Used to create charts and visual representations of the dataset.

Seaborn: Used to create statistical visualisations and explore patterns in the data.

Scikit-learn: Used for data preprocessing, splitting the dataset, implementing classification algorithms, and evaluating model performance.

2.9 Suitability of the Dataset

The selected dataset is suitable for this project because it contains both input attributes and an existing target variable. This structure supports supervised classification.

The dataset includes categorical and numerical information, allowing different preprocessing techniques to be demonstrated. It also provides an opportunity to perform exploratory data analysis and identify patterns across countries, sectors, and years.

The availability of the listing_status column makes it possible to train multiple classification algorithms and compare their performance using evaluation metrics such as accuracy, precision, recall, and F1-score.

2.10 Chapter Summary

This chapter described the IPO dataset used in the project, its source, structure, important attributes, and target variable. It also explained the selection of input features and the classification of attributes into numerical and categorical groups.

The dataset provides the foundation for the next stage of the project, which involves examining its characteristics through Exploratory Data Analysis (EDA), identifying patterns, and understanding the distribution of IPO records.

EXPLORATORY DATA ANALYSIS (EDA)

3.1 Introduction

Exploratory Data Analysis (EDA) is an important stage in a data science project. It involves examining, summarising, and visualising a dataset to understand its characteristics and identify meaningful patterns.

In this project, EDA is performed on the IPO tracking dataset to understand the distribution of IPO records across different listing statuses, years, countries, and business sectors.

Python libraries such as Pandas, Matplotlib, and Seaborn are used to perform the analysis and create visualisations. These visualisations help in understanding the dataset before applying machine learning classification algorithms.

3.2 Objectives of Exploratory Data Analysis

The main objectives of EDA in this project are:

1. To understand the overall structure and characteristics of the IPO dataset.
2. To examine the distribution of IPO listing status categories.
3. To analyse the number of IPO records across different years.
4. To identify the countries with the highest number of IPO records.
5. To identify the major business sectors represented in the dataset.
6. To observe patterns and trends that may be useful for machine learning classification.
7. To understand the numerical and categorical attributes available for model development.

3.3 Initial Dataset Inspection

The first step of EDA is to load the dataset and inspect its structure. The Pandas library is used to read the CSV file and examine the available records.

The following functions are used:

- head() – Displays the first five records of the dataset.
- shape – Returns the number of rows and columns.
- info() – Provides information about column names, data types, and non-null values.
- isnull().sum() – Identifies missing values in each column.
- duplicated().sum() – Counts duplicate records.

These functions help to understand the dataset and identify possible data-quality issues before further analysis.

3.4 Analysis of IPO Listing Status Distribution

The first visualisation examines the distribution of IPO records across different listing status categories.

A count plot is created using the listing_status column. It displays the number of records belonging to each status category.

This analysis helps to understand how the target variable is distributed within the dataset. It also provides an initial indication of whether the dataset contains balanced or imbalanced target classes.

Specific Requirements, Functions and Formulas

The listing_status column is selected to examine the distribution of IPO records across different status categories.

The following Python functions are used:

- value_counts() – Counts the number of records in each listing status category.
- plt.figure() – Creates the figure for visualization.
- sns.countplot() – Generates a count plot showing the frequency of each category.
- plt.xlabel() – Adds the X-axis label.
- plt.ylabel() – Adds the Y-axis label.
- plt.title() – Adds the chart title.

The frequency of each listing status is calculated using:

Frequency of a Listing Status = Total number of records belonging to that category.

Analysis Results

The graph shows that the Announced / Filed category has the highest number of records. The Completed category has the second-highest number of records, while Withdrawn / Postponed has considerably fewer records.

This indicates that the target variable is imbalanced. The difference in the number of records across categories is important when training and evaluating machine learning classification models.

Visualization

The distribution is represented using a count plot to compare the number of records in each IPO listing status category.

Observation:

The graph represents the frequency of each listing status category in the dataset. By comparing the heights of the bars, the distribution of the target classes can be examined. This information is useful when selecting suitable classification algorithms and evaluation metrics.

3.5 Analysis of IPO Records by Year

The second visualisation examines the distribution of IPO records across different years.

The year attribute is used to group the IPO records by year. A bar chart is generated to compare the number of records associated with each year.

This analysis helps identify the years with relatively higher or lower numbers of IPO records in the available dataset.

Specific Requirements, Functions and Formulas

The year attribute is used to examine the distribution of IPO records across different years.

The following Python functions are used:

- groupby() – Groups IPO records according to their year.
- size() – Calculates the number of records in each year.
- sort_index() – Arranges the yearly results in ascending order.
- plt.figure() – Creates the visualization area.
- plt.bar() – Generates a bar chart.
- plt.xlabel() and plt.ylabel() – Add axis labels.
- plt.title() – Adds the chart title.

The yearly record count is calculated using:

Number of IPO Records per Year = Count of records belonging to each year.

Analysis Results

The graph shows variations in the number of IPO records across the years represented in the dataset. Some years contain a higher number of records, while others contain comparatively fewer records.

The year-wise distribution helps identify changes in the availability of IPO records over time. These observations describe the dataset and do not independently establish the reasons behind yearly changes.

Visualization

A bar chart is used to compare the number of IPO records across different years.

Observation:

The visualisation provides a year-wise comparison of IPO records. It helps identify changes in the number of records across the years represented in the dataset. The graph describes the

distribution of available records and does not independently establish the reasons behind changes in IPO activity.

3.6 Analysis of Top 10 Countries by IPO Records

The third visualisation focuses on the geographical distribution of IPO records.

The country attribute is analysed to identify the ten countries with the highest number of IPO records. A bar chart is used to compare the number of records for these countries.

This analysis provides an overview of the geographical representation of IPO-related information in the dataset.

Specific Requirements, Functions and Formulas

The country attribute is selected to examine the geographical distribution of IPO records.

The following Python functions are used:

- value_counts() – Calculates the number of IPO records associated with each country.
- head(10) – Selects the ten countries with the highest record counts.
- plt.figure() – Creates the visualization area.
- plt.bar() – Generates a bar chart.
- plt.xlabel() and plt.ylabel() – Add axis labels.
- plt.title() – Adds the chart title.

The country-wise record count is calculated using:

Country-wise IPO Count = Total number of records associated with each country.

Analysis Results

The graph shows that the United States and India have the highest representation among the ten countries displayed. China and Canada follow, while Australia, the United Kingdom, Brazil, Germany, Hong Kong and Singapore also appear in the top ten.

This analysis highlights the geographical distribution of IPO records available in the dataset. The results represent the dataset's coverage and should not be treated as a complete measure of IPO activity in each country.

Visualization

A bar chart is used to compare the top ten countries according to their number of IPO records.

Observation:

The graph compares the ten countries with the highest number of IPO records. It helps identify which countries are more strongly represented in the available dataset. The results reflect the dataset's coverage and should not automatically be interpreted as the complete IPO activity of each country.

3.7 Analysis of Top 10 IPO Sectors

The fourth visualisation examines the distribution of IPO records across different business sectors.

The sector attribute is used to identify the ten sectors with the highest number of IPO records. A bar chart is generated to compare their frequencies.

This analysis helps understand the sector-wise composition of the dataset and identify the business sectors that are more frequently represented.

Specific Requirements, Functions and Formulas

The sector attribute is used to examine the distribution of IPO records across different business sectors.

The following Python functions are used:

- value_counts() – Counts the IPO records belonging to each sector.
- head(10) – Selects the ten sectors with the highest record counts.
- plt.figure() – Creates the visualization area.
- plt.bar() – Generates a bar chart.
- plt.xlabel() and plt.ylabel() – Add axis labels.
- plt.title() – Adds the chart title.

The sector-wise record count is calculated using:

Sector-wise IPO Count = Total number of records associated with each sector.

Analysis Results

The graph shows that Retail and Consumer Goods has the highest representation among the displayed sectors. Software and IT, Pharmaceuticals and Biotech, and Banking and Financial Services are also strongly represented.

Other sectors shown include Energy and Utilities, Healthcare, Other, Industrials and Manufacturing, Real Estate and Construction, and Mining and Metals.

The analysis provides an understanding of the industry-wise composition of the IPO dataset.

Visualization

A bar chart is used to compare the ten most frequently represented IPO sectors.

Observation:

The visualisation presents a comparison of the ten most frequently represented sectors. It helps identify the major sector categories available in the dataset and provides an understanding of its industry composition.

3.8 Time-Based Feature Extraction

The dataset contains a date-related attribute named first_report_date. This column is converted into a datetime format using Pandas.

Two additional features are extracted from this date:

- report_month: Represents the month of the first report.
- report_quarter: Represents the quarter of the first report.

These derived features provide additional time-related information for the machine learning models.

The extracted month and quarter are included among the numerical input features during model development.

3.9 Key Findings from EDA

The exploratory analysis provides an initial understanding of the IPO dataset.

The main findings examined through the visualisations are:

1. The distribution of IPO records across different listing status categories.
2. The year-wise distribution of available IPO records.
3. The geographical representation of IPO records across countries.
4. The sector-wise distribution of IPO records.
5. The availability of date-related information for extracting additional features.

The EDA stage helps understand the dataset and supports the selection of relevant attributes for machine learning.

3.10 Importance of EDA in Machine Learning

EDA plays an important role in preparing data for machine learning. It helps identify the structure of the dataset, understand the distribution of variables, and examine possible patterns.

In this project, EDA is used to understand the target variable and the input attributes before model development. The insights obtained from the visualisations also help explain the dataset and provide a foundation for the preprocessing stage.

3.11 Chapter Summary

This chapter presented the exploratory data analysis performed on the IPO tracking dataset. The analysis included initial dataset inspection, listing status distribution, year-wise IPO record distribution, country-wise analysis, and sector-wise analysis.

Time-based features were also extracted from the first report date. The results of EDA provide a better understanding of the dataset and prepare it for the next stage, which involves data preprocessing and machine learning model development.

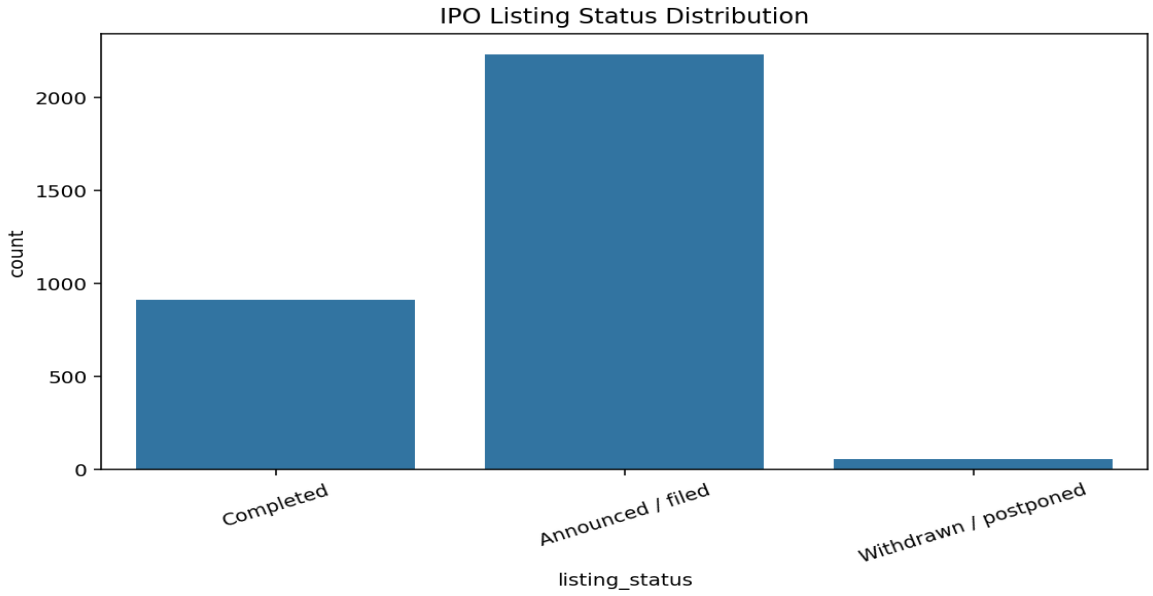


Figure 3.4: Distribution of IPO Listing Status

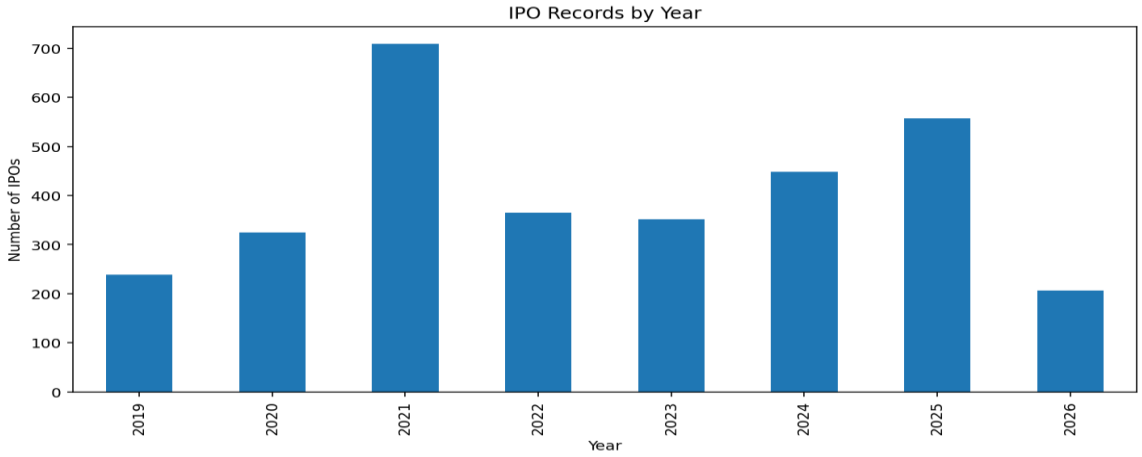


Figure 3.5: Number of IPO Records by Year

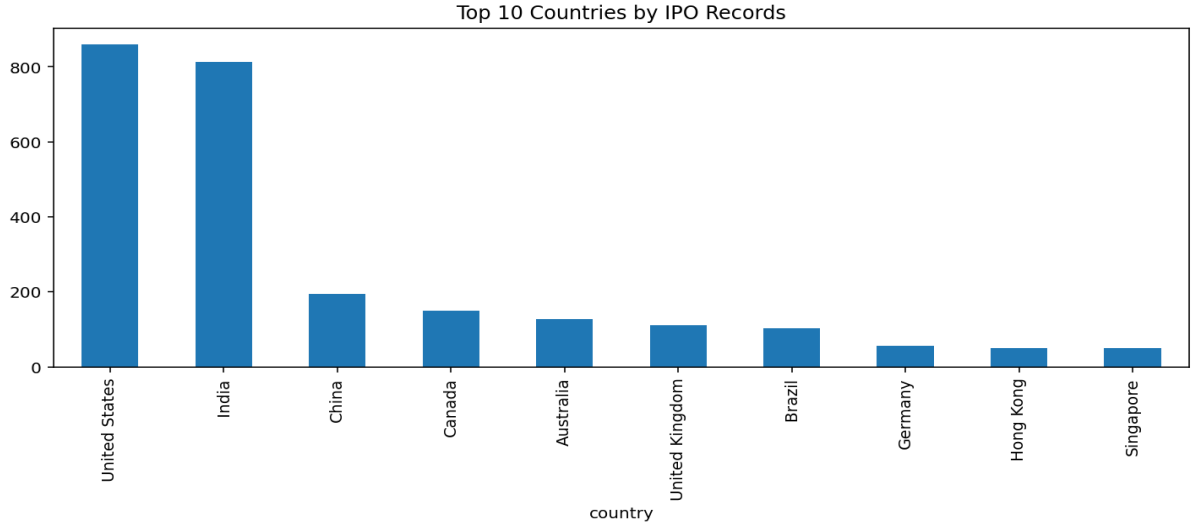


Figure 3.6: Top 10 Countries by IPO Records

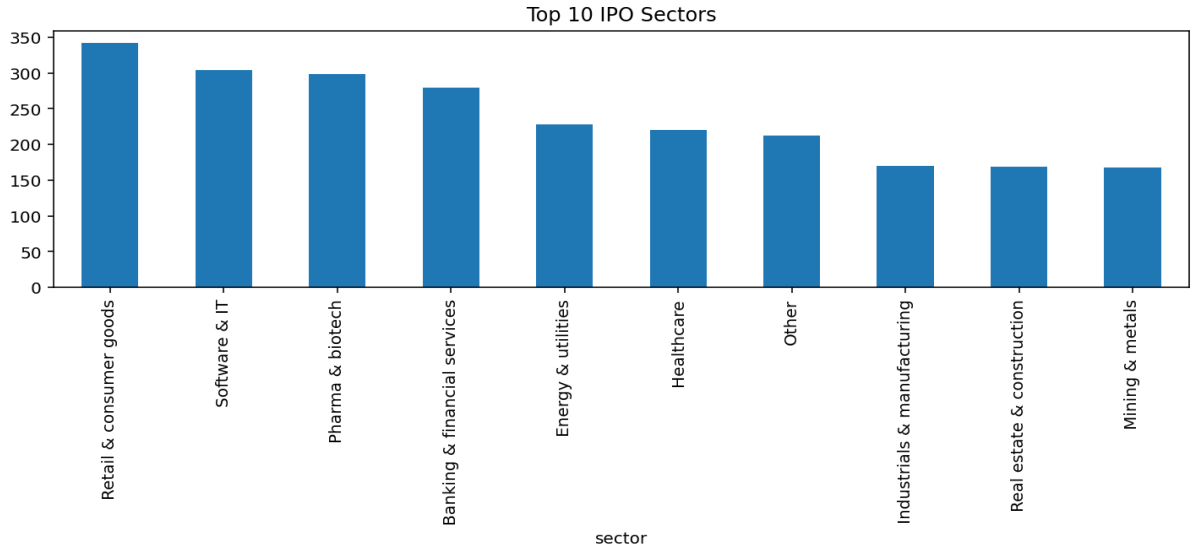


Figure 3.7: Top 10 IPO Sectors

DATA PREPROCESSING AND METHODOLOGY

4.1 Introduction

Data preprocessing is an important step in machine learning because raw datasets may contain missing values, duplicate records, inconsistent formats, and attributes that cannot be directly used by machine learning algorithms.

In this project, preprocessing is performed on the IPO tracking dataset before training the classification models. The main objective is to prepare the dataset in a suitable format while maintaining the relevant information required for prediction.

Python libraries, particularly Pandas and Scikit-learn, are used to perform the preprocessing operations.

4.2 Data Inspection

The dataset is first inspected to understand its structure and identify possible data-quality issues.

The following operations are performed:

- Displaying the first few records.
- Checking the number of rows and columns.
- Examining column names and data types.
- Identifying missing values.
- Checking duplicate records.

These operations help determine the preprocessing requirements of the dataset.

4.3 Date Conversion and Feature Extraction

The first_report_date column is converted into a datetime format using the Pandas library.

The conversion is performed using error handling so that invalid date values are treated as missing values.

Two additional features are extracted:

Report Month: Represents the month in which the first report was recorded.

Report Quarter: Represents the quarter associated with the first report.

These features provide additional time-related information that can be used during machine learning model training.

4.4 Selection of Input and Target Features

Feature selection involves identifying the attributes used as inputs for machine learning and the variable that the model must classify.

The following eight input features are selected:

1. ipo_type
2. year
3. news_reports
4. amount_status
5. country
6. sector
7. report_month
8. report_quarter

The target variable is listing_status.

The selected input features contain both numerical and categorical information. The target variable contains the existing IPO listing status categories.

4.5 Handling Missing Values

Missing values can affect the performance of machine learning algorithms. Therefore, an imputation technique is applied to handle missing values in the selected input features.

Two different strategies are used:

Numerical Features: Missing values are replaced using the median value of the respective numerical feature.

Categorical Features: Missing values are replaced using the most frequently occurring value in the respective categorical feature.

The SimpleImputer class from Scikit-learn is used to implement these operations.

4.6 Encoding Categorical Features

Machine learning algorithms generally require numerical input. Therefore, categorical features must be converted into a numerical representation.

One-Hot Encoding is applied to the categorical attributes:

- ipo_type
- amount_status
- country
- sector

The OneHotEncoder from Scikit-learn creates separate binary columns for the available categories.

The parameter handle_unknown="ignore" is used so that previously unseen categories do not cause an encoding error during transformation.

4.7 Feature Scaling

Feature scaling is applied to numerical attributes to bring them to a comparable scale.

In this project, StandardScaler is used for numerical features.

Standardisation transforms numerical values based on their mean and standard deviation. This helps prevent features with larger numerical ranges from dominating distance-based algorithms such as K-Nearest Neighbors.

The numerical features used are:

- year
- news_reports
- report_month
- report_quarter

4.8 Column Transformer

A ColumnTransformer is used to combine the preprocessing operations for numerical and categorical features.

The numerical preprocessing pipeline consists of median imputation followed by standardisation.

The categorical preprocessing pipeline consists of most-frequent-value imputation followed by one-hot encoding.

Combining these operations into a single preprocessing structure ensures that the appropriate transformation is applied to each feature group.

4.9 Train-Test Splitting

After defining the input features and target variable, the dataset is divided into training and testing subsets.

The train_test_split() function from Scikit-learn is used for this purpose.

The following configuration is used:

- Training dataset: 80%
- Testing dataset: 20%
- Random state: 42
- Stratification: Enabled using the target variable

Stratified splitting helps maintain approximately the same target-class proportions in both training and testing datasets.

The training subset is used to develop the classification models, while the testing subset is used to evaluate their performance on records not used for training.

4.10 Machine Learning Methodology

The project follows a supervised machine learning classification approach.

The methodology consists of the following stages:

1. Loading the IPO dataset.
2. Inspecting the dataset.
3. Converting the date column.
4. Extracting month and quarter features.
5. Selecting input and target variables.
6. Applying numerical and categorical preprocessing.
7. Splitting the dataset into training and testing subsets.
8. Training four classification algorithms.
9. Evaluating the models using classification metrics.
10. Comparing model performance and identifying the model with the highest macro F1-score.

4.11 Machine Learning Algorithms Selected

Four classification algorithms are implemented in this project.

4.11.1 Logistic Regression

Logistic Regression is a supervised classification algorithm used to estimate the probability of a record belonging to different target classes.

Although its name contains the word regression, it is commonly used for classification problems.

In this project, Logistic Regression is configured with a maximum of 2,000 iterations and balanced class weights. The balanced class-weight setting helps account for differences in the number of records belonging to each target category.

4.11.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a supervised machine learning algorithm that classifies a record based on the classes of its nearest neighbouring records.

The algorithm calculates distances between data points and uses the majority class among the selected neighbours to determine the classification.

In this project, the value of K is set to 5. Feature scaling is included in preprocessing because distance-based algorithms are sensitive to differences in feature scales.

4.11.3 Decision Tree Classifier

Decision Tree is a supervised learning algorithm that makes predictions using a tree-like structure of decision rules.

It divides the dataset into smaller groups based on feature conditions until a classification decision is reached.

In this project, the Decision Tree Classifier is configured with a random state of 42 and balanced class weights.

4.11.4 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees to produce a classification result.

Each tree contributes to the final prediction, and the class receiving the majority of votes is selected.

In this project, the Random Forest Classifier uses 100 decision trees, a random state of 42, and balanced class weights.

4.12 Pipeline Implementation

A machine learning pipeline combines preprocessing and model training into a single workflow.

In this project, a separate pipeline is created for each classification algorithm. The preprocessing operations are applied before the selected classifier.

The pipeline approach helps maintain consistency between training and testing transformations and reduces the possibility of data leakage during preprocessing.

4.13 Model Evaluation Approach

After training, each classification model is evaluated using the testing dataset.

The following evaluation measures are calculated:

- Accuracy

- Macro Precision
- Macro Recall
- Macro F1-score

The macro averaging method calculates the metric separately for each class and then takes the unweighted average across all classes.

The macro F1-score is particularly useful for comparing performance when the target classes are not equally represented.

The model with the highest macro F1-score is selected as the best-performing model among the four algorithms tested.

4.14 Chapter Summary

This chapter explained the data preprocessing and machine learning methodology used in the project. It covered date conversion, feature extraction, feature selection, missing-value imputation, categorical encoding, feature scaling, and train-test splitting.

It also introduced the four classification algorithms: Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest.

IMPLEMENTATION AND MODEL TRAINING

5.1 Introduction

Implementation is the stage where the proposed machine learning methodology is converted into a working Python program. In this project, the IPO dataset is processed and used to train four supervised machine learning classification algorithms.

The implementation is carried out using Python in the Spyder IDE. Scikit-learn is used to create preprocessing pipelines, train classification models, and evaluate their performance.

The main objective of this stage is to develop classification models that can classify IPO records according to their recorded listing status.

5.2 Development Environment

The project is implemented using the following software and libraries:

- Programming Language: Python
- IDE: Spyder
- Data Manipulation: Pandas and NumPy
- Data Visualisation: Matplotlib and Seaborn
- Machine Learning: Scikit-learn

These tools support dataset handling, preprocessing, model implementation, and performance evaluation.

5.3 Dataset Loading and Preparation

The IPO dataset is loaded into the Python environment using the Pandas library.

The CSV file is read into a DataFrame, which allows the records and attributes to be accessed and processed.

After loading the dataset, the date column is converted into datetime format. The month and quarter are extracted to create additional features.

The input features and target variable are then separated for model development.

5.4 Preprocessing Pipeline Implementation

A preprocessing pipeline is developed using Scikit-learn.

The numerical features are processed through median imputation and standardisation. The categorical features are processed through most-frequent-value imputation and one-hot encoding.

A ColumnTransformer combines both preprocessing workflows.

This preprocessing structure is integrated with each classification algorithm using the Pipeline class. As a result, the same preprocessing operations are applied consistently during model training and prediction.

5.5 Training and Testing Dataset

The prepared dataset is divided into training and testing subsets using an 80:20 ratio.

The training data is used to fit the classification models, while the testing data is used to evaluate their performance.

The random state is fixed at 42 to make the data split reproducible. Stratification is also applied using the target variable to maintain approximately similar class proportions in both subsets.

5.6 Implementation of Logistic Regression

Logistic Regression is implemented as the first classification algorithm.

The model is configured with a maximum of 2,000 iterations and balanced class weights.

The preprocessing pipeline is connected to the Logistic Regression classifier. The model is trained using the training dataset and evaluated using the testing dataset.

The algorithm estimates class probabilities and assigns each IPO record to a predicted listing status category.

5.7 Implementation of K-Nearest Neighbors

K-Nearest Neighbors is implemented as the second classification algorithm.

The number of neighbours is set to 5. The model uses the processed input features to calculate distances between records.

After training, the algorithm classifies testing records based on the majority class among their nearest neighbours.

Standardisation is particularly relevant for this algorithm because it relies on distance calculations.

5.8 Implementation of Decision Tree

Decision Tree Classifier is implemented as the third algorithm.

The model uses feature-based decision rules to divide records into different groups. These divisions help the classifier determine the target category for each record.

The model is configured with a random state of 42 and balanced class weights.

The trained model is then used to predict the listing status of the testing records.

5.9 Implementation of Random Forest

Random Forest Classifier is implemented as the fourth algorithm.

The model consists of 100 decision trees. Each tree contributes to the final classification, and the final output is determined through majority voting.

A random state of 42 and balanced class weights are used in the model configuration.

Random Forest combines multiple decision trees to produce a classification result and is evaluated using the same testing dataset as the other models.

5.10 Model Training Process

All four classification models are trained using their respective preprocessing and classification pipelines.

The training process involves:

1. Passing the training features into the pipeline.
2. Applying the required preprocessing transformations.
3. Training the selected classification algorithm.
4. Generating predictions using the testing features.
5. Calculating the evaluation metrics.

Using the same training and testing split allows the performance of the four algorithms to be compared under consistent conditions.

5.11 Prediction Generation

After training, each model generates predictions for the testing dataset.

The predicted values represent the listing status categories assigned by the model.

These predictions are compared with the actual target values to calculate the evaluation metrics.

The results are stored for further comparison and analysis.

5.12 Model Results Export

The evaluation results of the four classification algorithms are collected into a structured results table.

The table contains the model name and the calculated evaluation metrics, including accuracy, macro precision, macro recall, and macro F1-score.

The results are exported into a CSV file named:

IPO_Model_Results_Final.csv

This file provides a convenient way to review and compare the performance of the implemented models.

5.13 Best Model Selection

The best-performing model is selected based on the highest macro F1-score.

Macro F1-score gives equal importance to each target class when calculating the average F1-score.

This selection approach is useful for the project because the listing status categories may have different numbers of records.

The selected model is then used for further evaluation through a confusion matrix and classification report.

5.14 Chapter Summary

This chapter explained the practical implementation of the IPO listing status classification project using Python and Spyder.

Four classification algorithms were implemented: Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest.

The models were trained using a common preprocessing workflow and training dataset. Their predictions were evaluated using accuracy, macro precision, macro recall, and macro F1-score.

The results were exported for comparison, and the model with the highest macro F1-score was selected for further evaluation. The next chapter presents the experimental results and performance analysis.

RESULTS AND PERFORMANCE ANALYSIS:

6.1 Introduction

After implementing and training the four machine learning classification algorithms, their performance is evaluated using the testing dataset.

The main objective of this chapter is to analyse and compare the performance of Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest.

The evaluation is performed using four important classification metrics: accuracy, precision, recall, and F1-score. Graphical comparisons are also used to understand the differences in performance between the four models.

A confusion matrix is generated for the selected model to examine the actual and predicted IPO listing status categories.

6.2 Experimental Results

The four classification algorithms are trained using the same training dataset and evaluated using the testing dataset.

The models generate predictions for three IPO listing status categories:

- Announced / filed
- Completed
- Withdrawn / postponed

The predicted results are compared with the actual target values to calculate the evaluation metrics.

The experimental results are used to compare the classification performance of the four algorithms and identify the model with the strongest overall performance.

6.3 Evaluation Metrics

The following evaluation metrics are used in this project.

6.3.1 Accuracy

Accuracy measures the proportion of correctly classified records out of the total number of testing records.

It provides an overall indication of how accurately a model classifies the testing dataset.

6.3.2 Precision

Precision measures the proportion of correctly predicted records among all records predicted to belong to a particular class.

It indicates how reliable the model's predictions are for each listing status category.

6.3.3 Recall

Recall measures the proportion of actual records belonging to a particular class that are correctly identified by the model.

It helps evaluate the model's ability to identify records from each listing status category.

6.3.4 F1-Score

F1-score is the harmonic mean of precision and recall. It provides a combined measure of these two evaluation metrics.

Macro F1-score calculates the F1-score separately for each class and then takes the unweighted average. This is useful when the target variable contains classes with different numbers of records.

6.4 Model Performance Comparison

Four classification algorithms are evaluated using accuracy, macro precision, macro recall, and macro F1-score.

The comparison helps identify the strengths and limitations of each algorithm.

.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-score
Logistic Regression	**0.6720**	**0.5419**	**0.6420**	**0.5111**
K-Nearest Neighbors	**0.8460**	**0.5440**	**0.4921**	**0.5094**
Decision Tree	**0.4770**	**0.4918**	**0.5616**	**0.4168**
Random Forest	**0.8260**	**0.5926**	**0.7006**	**0.6073**

Random Forest achieved the highest macro F1-score of 0.6073 among the four evaluated models and was therefore selected as the best-performing model. Although K-Nearest Neighbors achieved the highest accuracy of 0.8460 (84.60%), Random Forest provided better overall performance across the three classes based on macro precision, macro recall, and macro F1-score.

The values should be entered directly from the IPO_Model_Results_Final.csv file.

6.5 Accuracy Comparison

Accuracy comparison is used to examine the overall classification performance of the four machine learning models.

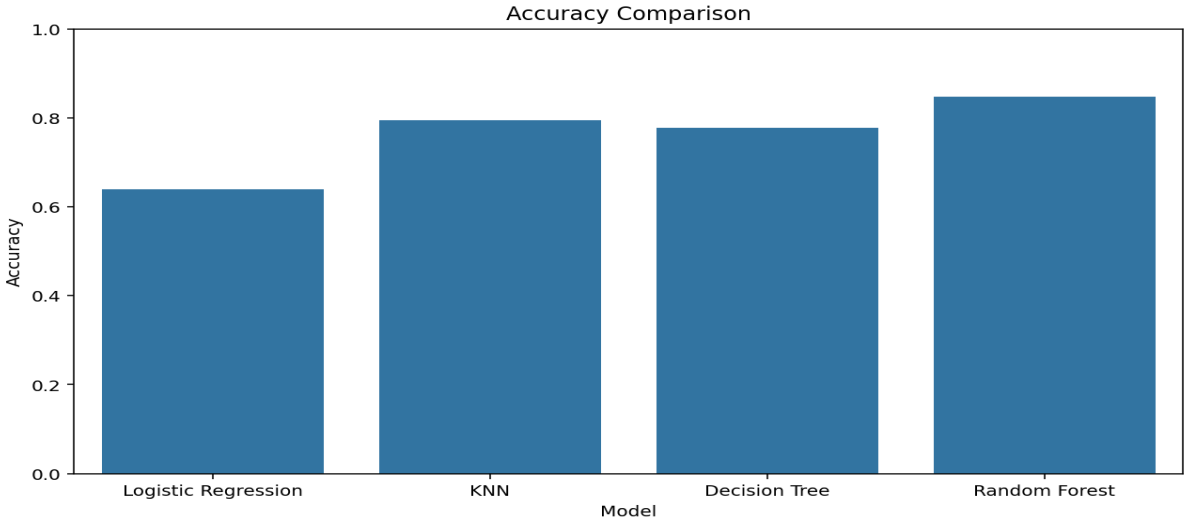


Figure 6.1: Accuracy Comparison

Observation:

The graph compares the accuracy of Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest. Random Forest has the highest accuracy among the four models, while Logistic Regression has the lowest accuracy in the displayed comparison.

This indicates that Random Forest correctly classifies a larger proportion of testing records than the other models in this experiment.

6.6 Precision Comparison

Precision comparison helps evaluate how reliably each model predicts the IPO listing status categories.

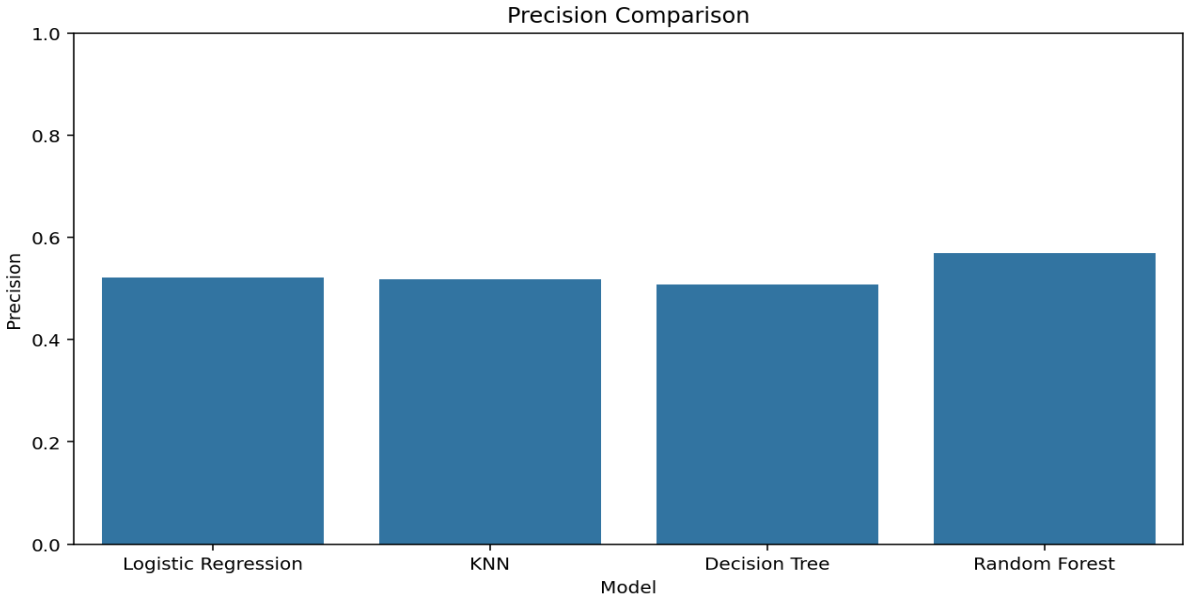


Figure 6.2: Precision Comparison

Observation:

The graph shows the precision scores of the four classification algorithms. Random Forest achieves the highest precision, while Decision Tree has the lowest precision in the displayed comparison.

This indicates that Random Forest provides better precision performance among the tested algorithms.

6.7 Recall Comparison

Recall comparison examines the ability of each model to identify actual records belonging to the different listing status categories.

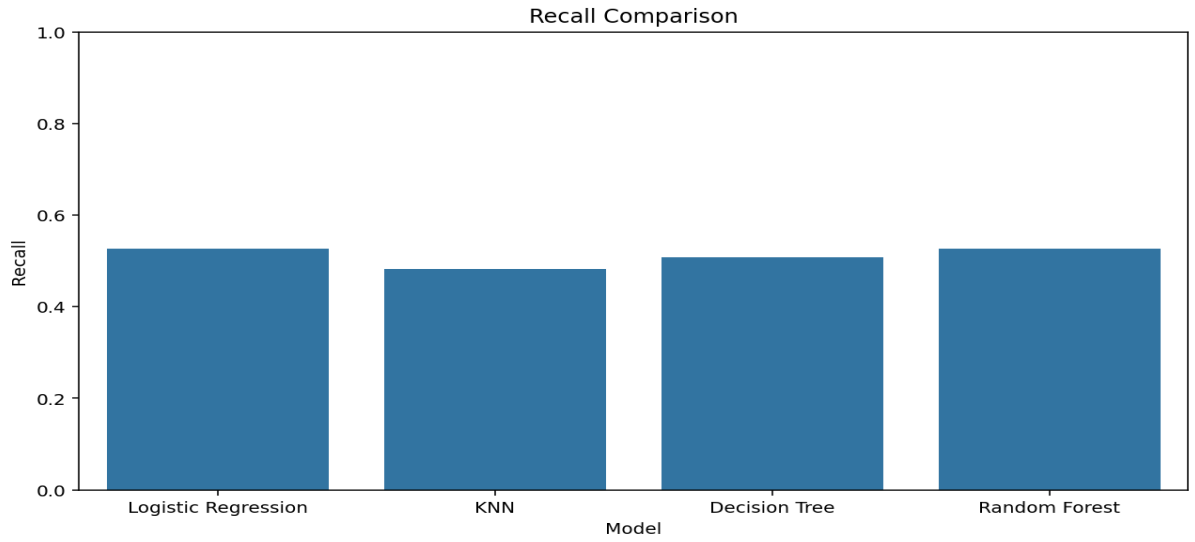


Figure 6.3: Recall Comparison

Observation:

The graph compares the recall scores of the four models. Random Forest achieves the highest recall, while K-Nearest Neighbors shows the lowest recall in the displayed comparison.

A higher recall indicates a better ability to identify actual records belonging to the target categories.

6.8 F1-Score Comparison

F1-score provides a combined evaluation of precision and recall. It is useful for comparing classification models when the target categories are not equally distributed.

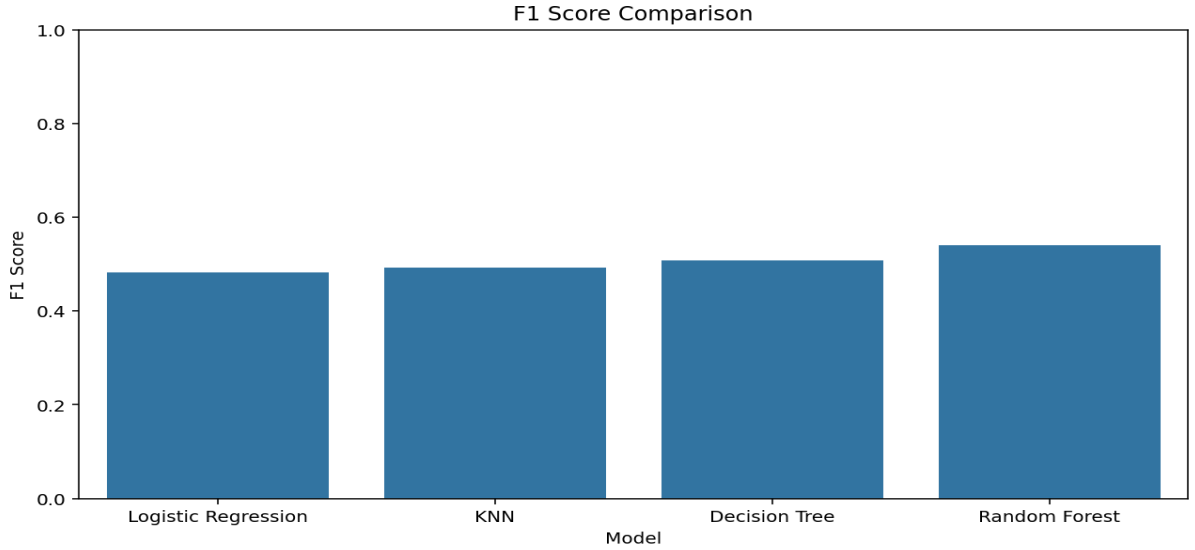


Figure 6.4: F1 Score Comparison

Observation:

The graph shows that Random Forest achieves the highest F1-score among the four classification algorithms. Logistic Regression, K-Nearest Neighbors, and Decision Tree have comparatively lower F1-scores.

Based on this comparison, Random Forest provides the strongest balance between precision and recall among the tested models.

6.9 Confusion Matrix Analysis

A confusion matrix is used to compare the actual listing status categories with the categories predicted by the selected classification model.

It provides detailed information about correctly classified and incorrectly classified records.

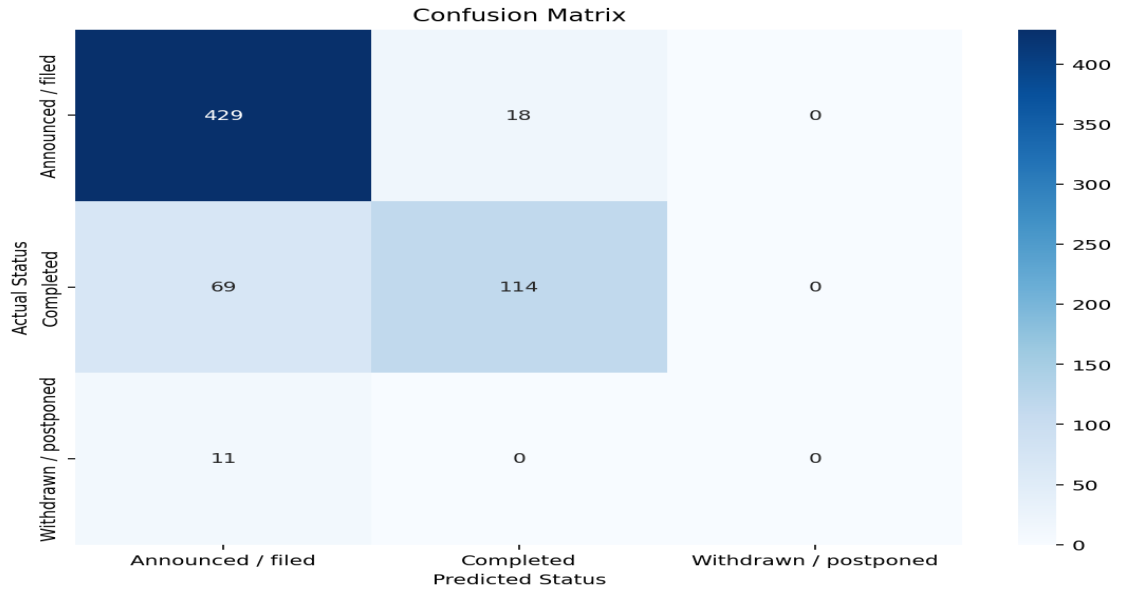


Figure 6.5: Confusion Matrix

Observation:

The confusion matrix displays the classification results for three IPO listing status categories.

The model correctly classified 429 records belonging to the Announced / filed category and 114 records belonging to the Completed category.

However, 18 Announced / filed records were incorrectly classified as Completed, while 69 Completed records were incorrectly classified as Announced / filed.

The model did not correctly identify any records from the Withdrawn / postponed category. All 11 actual records in this category were classified as Announced / filed.

The confusion matrix indicates that the model performs better on the Announced / filed category than on the other categories. It also highlights the difficulty of classifying the Withdrawn / postponed category, which has very few records in the testing dataset.

6.10 Best-Performing Model Selection

The four classification algorithms are compared using their evaluation metrics. The macro F1-score is used as the main criterion for selecting the best-performing model.

Based on the F1-score comparison graph, Random Forest achieves the highest F1-score.

Best Model: Random Forest

Macro F1-score:

Random Forest is therefore selected as the best-performing model based on the displayed F1-score comparison.

6.11 Interpretation of Results

The experimental results demonstrate that different machine learning algorithms produce different classification performances on the IPO dataset.

Random Forest performs best in the displayed accuracy, precision, recall, and F1-score comparisons.

The confusion matrix provides further insight into the classification results. It shows that the model can identify many Announced / filed and Completed records but has difficulty identifying Withdrawn / postponed records.

One important factor is the imbalance in the target variable. The dataset contains substantially fewer Withdrawn / postponed records than the other categories. This can affect the model's ability to learn and classify the less-represented category.

Therefore, accuracy alone is not sufficient to evaluate the model. Precision, recall, macro F1-score, and the confusion matrix are also important for understanding its performance.

6.12 Limitations of the Experimental Results

The results depend on the quality, size, and distribution of the available IPO dataset.

The classification models are evaluated using a testing subset of the supplied dataset. Therefore, the results represent performance on this particular experimental dataset and may differ when applied to new or external records.

The model classifies the recorded IPO listing status categories. It does not predict future IPO share prices, listing gains, or investment returns.

The limited number of Withdrawn / postponed records also affects the model's ability to identify that category accurately.

6.13 Chapter Summary

This chapter presented the experimental results and performance analysis of four machine learning classification algorithms.

The models were compared using accuracy, precision, recall, and F1-score. Separate graphs were used to visualise the performance of each evaluation metric.

The confusion matrix provided a detailed analysis of actual and predicted listing status categories. The results showed that Random Forest achieved the strongest performance in the displayed metric comparisons and was selected as the best-performing model.

The next chapter presents the overall conclusion and possible future improvements to the project.

7. CONCLUSION AND FUTURE SCOPE:

7.1 Introduction

This chapter presents the overall conclusion of the project, summarising the work carried out, the machine learning techniques implemented, and the findings obtained from the experimental results.

The project focuses on applying data science and machine learning classification techniques to analyse IPO records and classify their recorded listing status.

7.2 Conclusion

The project, titled "IPO Listing Status Prediction Using Machine Learning Classification", was developed to apply machine learning techniques to historical IPO data.

The project involved dataset inspection, data preprocessing, exploratory data analysis, feature selection, model development, and performance evaluation.

Four supervised machine learning classification algorithms were implemented: Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest.

The dataset was preprocessed using missing-value imputation, One-Hot Encoding, and feature scaling. Additional time-related features, including report month and report quarter, were extracted from the date attribute.

The four classification models were evaluated using accuracy, macro precision, macro recall, and macro F1-score. Graphical comparisons and a confusion matrix were used to understand the performance of the models.

Based on the displayed F1-score comparison, Random Forest achieved the highest F1-score among the four implemented algorithms. Therefore, it was selected as the best-performing model in this experiment.

The confusion matrix also highlighted the difficulty of correctly classifying the Withdrawn / postponed category because of its limited representation in the testing dataset.

Overall, the project demonstrates how Python and machine learning algorithms can be used to analyse IPO records and classify their recorded listing status.

7.3 Objectives Achieved

The major objectives of the project were successfully addressed through the implementation process.

The following objectives were achieved:

1. Studied and analysed the available IPO dataset.
2. Performed exploratory data analysis using graphical visualisations.

3. Identified relevant input features and the target variable.
4. Applied data preprocessing techniques to prepare the dataset.
5. Implemented four machine learning classification algorithms.
6. Evaluated the models using multiple classification metrics.
7. Compared the performance of the models through graphical representations.
8. Selected the best-performing model based on macro F1-score.

7.4 Applications of the Project

The project demonstrates the use of machine learning classification in financial dataset analysis.

Potential applications include:

- IPO Data Analysis: Organising and analysing historical IPO records.
- Listing Status Classification: Classifying IPO records into their recorded listing status categories.
- Financial Data Research: Supporting academic research involving IPO-related datasets.
- Exploratory Analysis: Understanding IPO records based on attributes such as country, sector, year, and report count.
- Machine Learning Education: Demonstrating the practical implementation of classification algorithms.

The project is intended for academic analysis and does not provide investment recommendations.

7.5 Limitations of the Project

Although the project successfully implements machine learning classification, certain limitations remain.

1. Dataset Dependency: The results depend on the quality and completeness of the available IPO dataset.
2. Class Imbalance: The number of records in the three listing status categories is not equal.
3. Limited Minority-Class Recognition: The confusion matrix indicates difficulty in correctly identifying Withdrawn / postponed records.
4. Feature Availability: Model performance depends on the input features available in the dataset.
5. Testing Scope: The models are evaluated using a testing subset of the supplied dataset, and their performance on external data may differ.
6. Prediction Scope: The project classifies recorded listing status categories and does not predict IPO share prices, listing gains, or investment returns.

7.6 Future Scope

The project can be improved and extended in several ways.

7.6.1 Dataset Expansion

Future work can include a larger IPO dataset containing more historical records and additional IPO-related attributes. A larger and more representative dataset may help improve classification performance.

7.6.2 Class Imbalance Handling

Advanced techniques such as SMOTE, random oversampling, and random undersampling can be explored to address the imbalance between listing status categories.

7.6.3 Advanced Machine Learning Algorithms

Additional classification algorithms, such as Support Vector Machine, Gradient Boosting, and XGBoost, can be tested and compared with the existing models.

7.6.4 Hyperparameter Tuning

Techniques such as Grid Search and Randomized Search can be applied to identify suitable model parameters and potentially improve classification performance.

7.6.5 Feature Engineering

Additional relevant features can be explored to improve the information available to the classification models. These may include IPO issue size, subscription information, industry characteristics, and other available historical attributes.

7.6.6 Web-Based Application

The selected classification model can be integrated into a web-based application. Users could enter the required IPO attributes and receive a predicted listing status category.

7.6.7 Model Deployment

The trained model can be deployed using frameworks such as Flask or Streamlit. This would make the classification system accessible through a user-friendly interface.

7.7 Final Conclusion

The project provided practical experience in applying data science and machine learning techniques to a financial dataset.

It covered the complete machine learning workflow, starting from dataset preparation and exploratory analysis to model training, evaluation, and comparison.

The implementation of four classification algorithms helped demonstrate the differences between machine learning approaches. Based on the F1-score comparison, Random Forest was selected as the best-performing model in the experiment.

The project also highlighted the importance of data quality, feature preparation, class distribution, and appropriate evaluation metrics in developing classification systems.

This work establishes a foundation for further exploration of machine learning applications in IPO data analysis.

7.8 Chapter Summary

This chapter presented the conclusion, objectives achieved, applications, limitations, and future scope of the project.

The project successfully demonstrated the implementation and comparison of four machine learning classification algorithms for classifying IPO listing status records. Future improvements can focus on expanding the dataset, addressing class imbalance, testing advanced algorithms, and developing a web-based application.

REFERENCES

1. Scikit-learn Developers, "Scikit-learn: Machine Learning in Python." Available: https://scikit-learn.org/stable/
2. The Pandas Development Team, "Pandas: Python Data Analysis Library." Available: https://pandas.pydata.org/docs/
3. NumPy Developers, "NumPy Documentation." Available: https://numpy.org/doc/
4. Matplotlib Development Team, "Matplotlib: Visualization with Python." Available: https://matplotlib.org/stable/
5. Seaborn Developers, "Seaborn: Statistical Data Visualization." Available: https://seaborn.pydata.org/
6. Scikit-learn Developers, "Classification Metrics." Available: https://scikit-learn.org/stable/modules/model_evaluation.html
7. Scikit-learn Developers, "Preprocessing Data." Available: https://scikit-learn.org/stable/modules/preprocessing.html
8. Scikit-learn Developers, "Pipeline and Composite Estimators." Available: https://scikit-learn.org/stable/modules/compose.html
9. Scikit-learn Developers, "Logistic Regression." Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
10. Scikit-learn Developers, "Nearest Neighbors Classification." Available: https://scikit-learn.org/stable/modules/neighbors.html
11. Scikit-learn Developers, "Decision Trees." Available: https://scikit-learn.org/stable/modules/tree.html
12. Scikit-learn Developers, "Random Forest Classifier." Available: https://scikit-learn.org/stable/modules/ensemble.html#random-forests
13. IPO Dataset, "IPO Tracking Dataset (2019–2026)." Dataset source: [Insert the original dataset website or download link].

APPENDIX A: PYTHON SOURCE CODE

This appendix contains the Python source code developed for the project titled "IPO Listing Status Prediction Using Machine Learning Classification".

The implementation was carried out using the Spyder IDE. Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn were used for data handling, visualisation, preprocessing, model development, and evaluation.

The complete source code includes the following major operations:

1. Importing the required Python libraries.
2. Loading the IPO dataset.
3. Inspecting and preprocessing the dataset.
4. Performing exploratory data analysis.
5. Selecting input features and the target variable.
6. Creating numerical and categorical preprocessing pipelines.
7. Splitting the dataset into training and testing subsets.
8. Implementing Logistic Regression, K-Nearest Neighbors, Decision Tree, and Random Forest.
9. Evaluating the classification models.
10. Generating performance comparison graphs and a confusion matrix.
11. Exporting the final model evaluation results.

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer


from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

df = pd.read_csv("C:/Users/ambati saketh reddy/Downloads/ipo_data_rows.csv")

print("Dataset Shape:", df.shape)
print(df.head())
print(df.info())
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())

df["first_report_date"] = pd.to_datetime(df["first_report_date"], errors="coerce")
df["report_month"] = df["first_report_date"].dt.month.fillna(0).astype(int)
df["report_quarter"] = df["first_report_date"].dt.quarter.fillna(0).astype(int)

plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="listing_status")
plt.title("IPO Listing Status Distribution")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
df["year"].value_counts().sort_index().plot(kind="bar")
plt.title("IPO Records by Year")
plt.xlabel("Year")
plt.ylabel("Number of IPOs")
```

```
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
df["country"].value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Countries by IPO Records")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
df["sector"].value_counts().head(10).plot(kind="bar")
plt.title("Top 10 IPO Sectors")
plt.tight_layout()
plt.show()

features = [
    "ipo_type",
    "year",
    "news_reports",
    "amount_status",
    "country",
    "sector",
    "report_month",
    "report_quarter"
]

X = df[features]
y = df["listing_status"]

categorical_features = [
```

```
    "ipo_type",
    "amount_status",
    "country",
    "sector"
]

numerical_features = [
    "year",
    "news_reports",
    "report_month",
    "report_quarter"
]

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numerical_features),

    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_features)
])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000,
        class_weight="balanced"
    ),

    "KNN": KNeighborsClassifier(
        n_neighbors=5
    ),

    "Decision Tree": DecisionTreeClassifier(
        random_state=42,
        class_weight="balanced"
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        class_weight="balanced"
    )
}

results = []
trained_models = {}

for name, model in models.items():
```

```
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", model)
    ])

    pipeline.fit(X_train, y_train)

    predictions = pipeline.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
    precision = precision_score(y_test, predictions, average="macro", zero_division=0)
    recall = recall_score(y_test, predictions, average="macro", zero_division=0)
    f1 = f1_score(y_test, predictions, average="macro", zero_division=0)

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1 Score": f1
    })

    trained_models[name] = pipeline

    print("\n", name)
    print(classification_report(y_test, predictions, zero_division=0))

results_df = pd.DataFrame(results)

print("\nMODEL PERFORMANCE COMPARISON")
```

```
print(results_df.round(4))

best_model_name = results_df.sort_values(
    by="F1 Score",
    ascending=False
).iloc[0]["Model"]

best_model = trained_models[best_model_name]
best_predictions = best_model.predict(X_test)

print("\nBest Model:", best_model_name)

cm = confusion_matrix(
    y_test,
    best_predictions,
    labels=best_model.classes_
)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=best_model.classes_,
    yticklabels=best_model.classes_
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Status")
plt.ylabel("Actual Status")
```

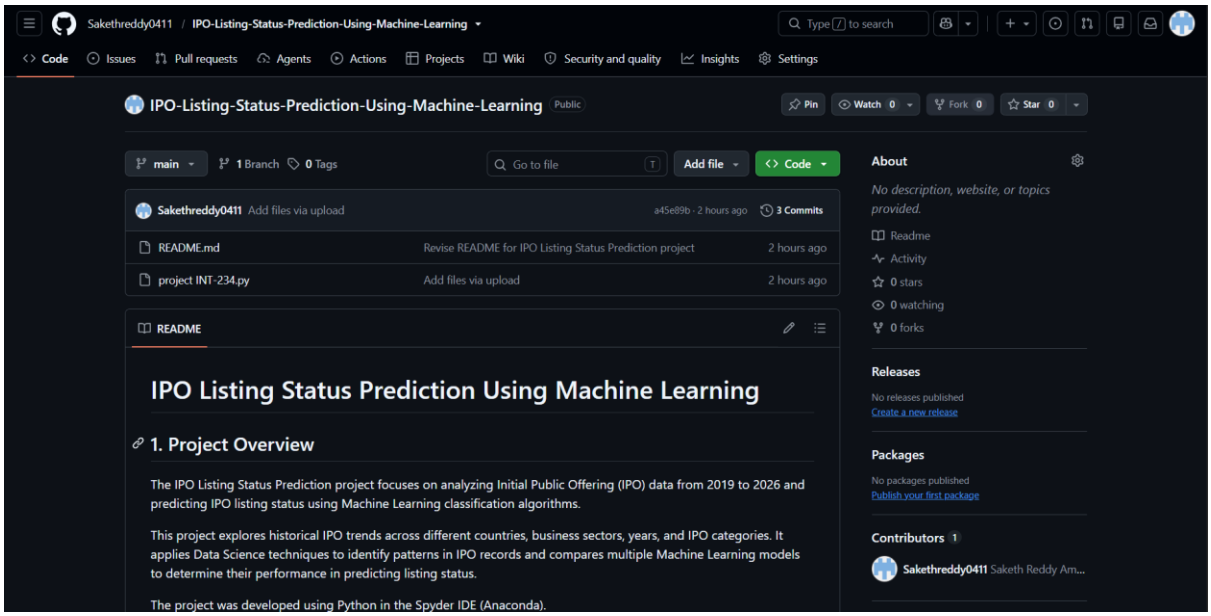
```
plt.tight_layout()
plt.show()

for metric in ["Accuracy", "Precision", "Recall", "F1 Score"]:
    plt.figure(figsize=(9, 5))
    sns.barplot(data=results_df, x="Model", y=metric)
    plt.title(metric + " Comparison")
    plt.ylim(0, 1)
    plt.tight_layout()
    plt.show()

results_df.to_csv(
    r"C:\Users\ambati saketh reddy\Downloads\IPO_Model_Results_Final.csv",
    index=False
)
print("CSV file saved successfully!")
```

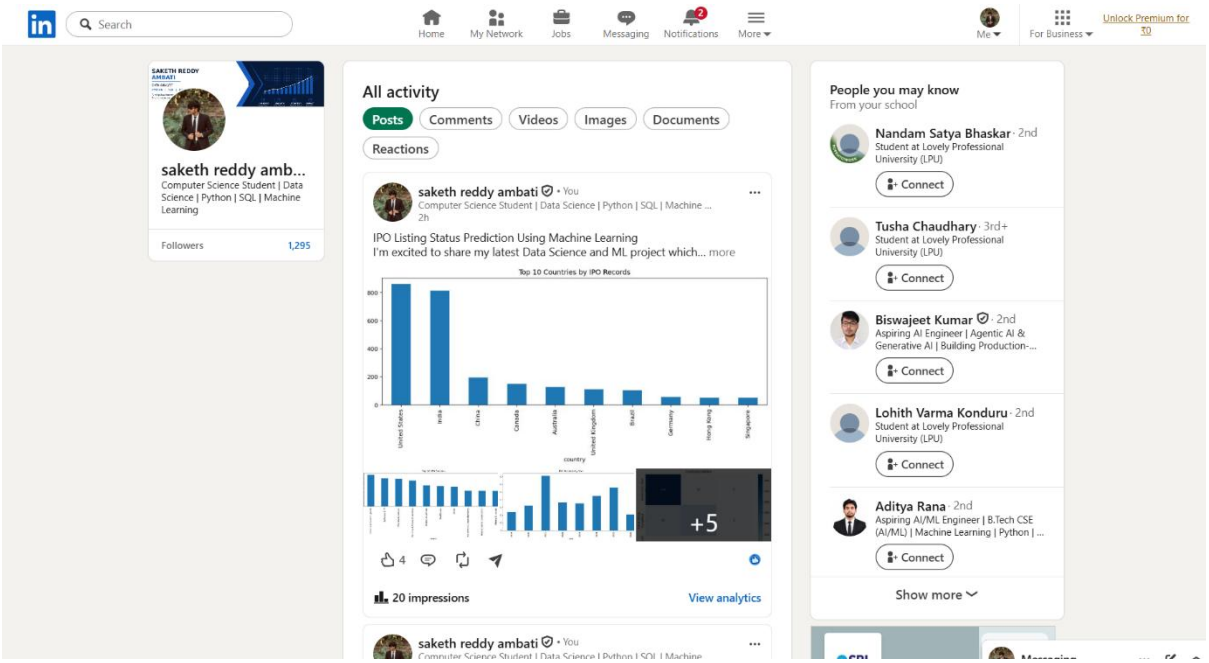
APPENDIX B: PROJECT LINKS

C.1 GitHub Repository



Link: https://github.com/Sakethreddy0411/

C.2 LinkedIn Project Post



Link: https://lnkd.in/p/

C.3 Development Environment

The following tools and technologies were used for developing and implementing the IPO Listing Status Prediction project:

- **Integrated Development Environment (IDE):** Spyder
- **Programming Language:** Python
- **Machine Learning Library:** Scikit-learn